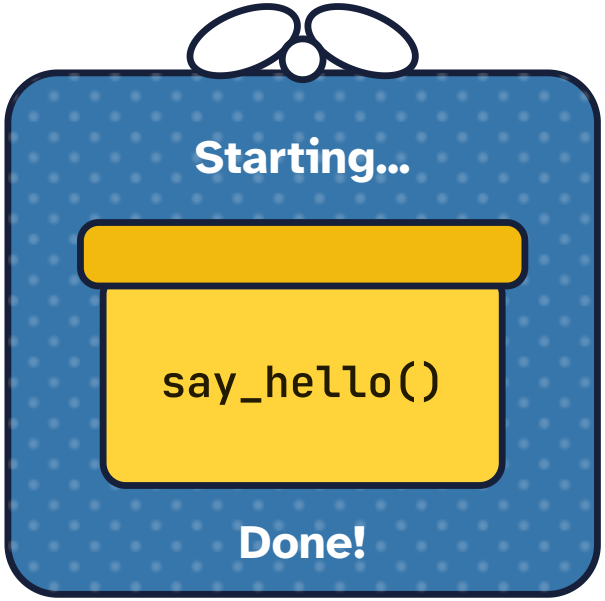
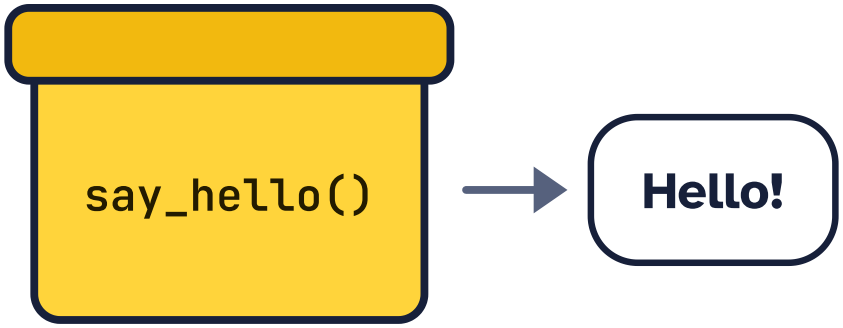
PYTHON, EXPLAINED LIKE YOU'RE FIVE

A decorator is gift wrap for a function.

It adds extra steps before and after a function runs. The function itself stays the same.

- Blue = the decorator (the wrap)
- Yellow = your function (the gift)





1

A function is a box that does one job.

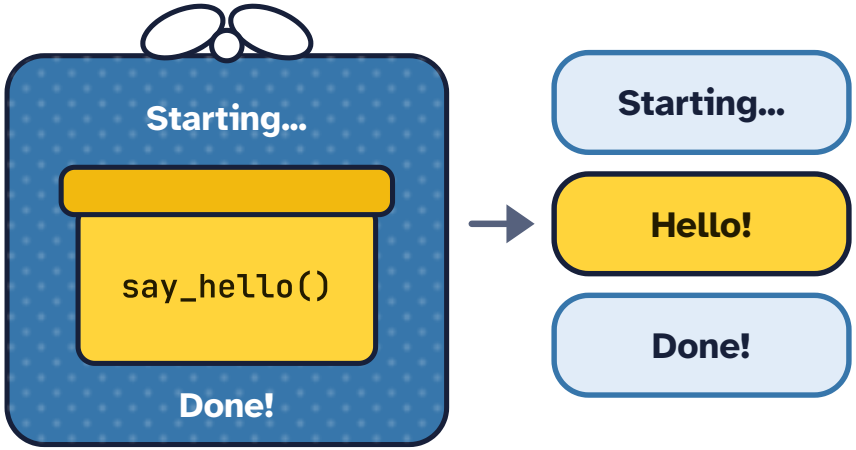
Call `say_hello()` and out comes "Hello!".



2

A decorator is wrapping paper with extra jobs on it.

One job for before the box opens. One job for after.



3

Wrap the box. Now one call does three things.

The box inside never changed. The wrapping added the extras around it.

4

Try it: put the wrap on, take it off.

Tap a button and watch what gets printed.

demo.py

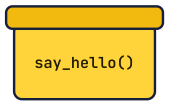
Copy code

```python
def announce(func):            # the wrapping paper
    def wrapper():
        print("Starting...")   # before
        func()                 # the gift
        print("Done!")         # after
    return wrapper

@announce                      # put the wrap on
def say_hello():
    print("Hello!")

say_hello()
```

With @announce Without




```
$ python demo.py
Hello!
```

No wrap, so only the gift runs.

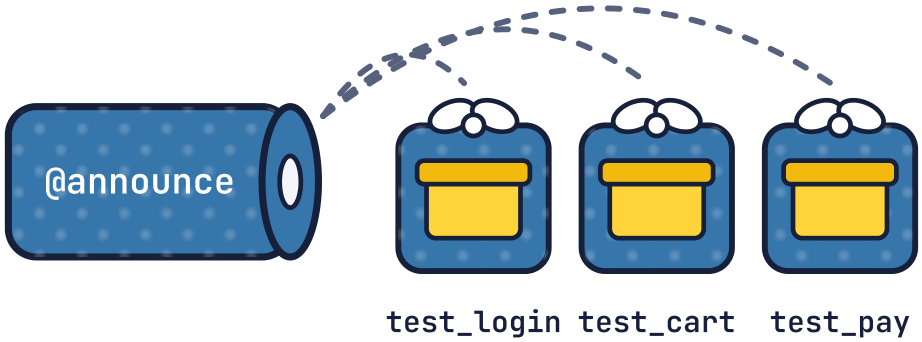
The @ line is a shortcut. These two do the same thing:

```python
@announce
def say_hello():
    ...
```

=

```python
def say_hello():
    ...
say_hello = announce(say_hello)
```

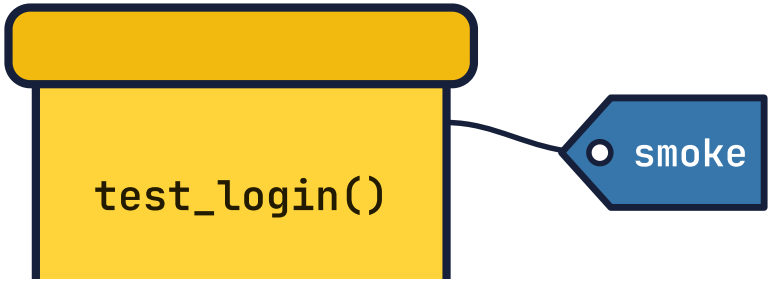
Python hands `say_hello` to `announce`, then keeps the wrapped version under the same name.



5

Write the wrap once. Use it on every box.

One `@announce` line adds the extras to any function. Handy for logging, timing and retries.



6

You already use decorators in pytest.

Some decorators just tie a label on. `@pytest.mark.smoke` tags the test, so `pytest -m smoke` runs only tagged tests.

```
import pytest
```

```
@pytest.mark.smoke
def test_login():
    assert 1 + 1 == 2
```

REMEMBER THIS ONE LINE

A decorator wraps a function to add steps before and after it, without editing the function.

Next level: functions with inputs

Let the wrapper take `*args, **kwargs` and pass them straight through.

```
def announce(func):
    def wrapper(*args, **kwargs):
        print("Starting...")
        result = func(*args, **kwargs)
        print("Done!")
        return result
    return wrapper
```